

Project in Model-Based Machine Learning by Group 25

HMM for Stock Market Prediction

Patrick Jensen Martins (s204748), Lydia Kasapi (s233564), Rita Saraiva (s205717), Afonso Cunha (s233177)

Abstract

The stock market is a complex and dynamic system, making the task of predicting the future price of a stock challenging. This volatility can be overcome by implementing models capable of capturing temporal dependencies and latent structure in data. In this project, we investigate the application of **Hidden Markov Models (HMMs)** for stock price prediction using real-world financial time series - the stock price in the period of 2017 to 2024 for three publicly traded companies: Apple (AAPL), IBM, and Dell (DELL). Inspired by previous work in the literature, combined with knowledge gained from the Model-Based Machine Learning course, we developed our implementation in Python which follows a model-driven design. The performance of the model was measured to evaluate prediction accuracy and ability to capture the correct movement direction. The results provide insights into the applicability of HMMs in modeling and forecasting stock behavior based on recent historical patterns.

Index Terms— Hidden Markov Models, Stock Market Prediction, Time Series Modeling, Forecasting, Financial Data.

1 Introduction

1.1 Motivation

The stock market is a highly dynamic environment influenced by countless macroeconomic, political, and behavioral factors. For decades, both institutional and retail investors have sought to develop accurate models that can predict stock price movements and thereby improve investment decisions.

1.2 Related Work

Among various statistical and machine learning techniques, **Hidden Markov Models (HMMs)** have emerged as a powerful framework for capturing the underlying temporal dependencies and structural patterns in financial time series data; aligning well with the nature of financial markets, where trends often reflect hidden investor sentiment, or macroeconomic factors. Several studies have explored the use of HMMs for stock market forecasting, ranging from basic discrete-state models to more sophisticated extensions such as Gaussian mixtures. A recent contribution we draw inspiration from is the work by [Catello et al. \(2023\)](#).

While we take methodological inspiration from their work, our aim is not to replicate it exactly but

to explore the application of HMMs using a model-driven approach in Python. Our dataset consists of daily stock prices for three major U.S.-listed companies — Apple, IBM, and Dell— spanning from 2017 to 2024. For each trading day, we derive a set of fractional features that describe the day’s price movement. These continuous features are discretized and modeled using a multivariate discrete HMM framework.

1.3 Research Questions

Can a Hidden Markov Model trained on historical fractional stock price patterns effectively predict future daily price movements for large-cap U.S. stocks?

To assess whether we have reached this goal, we evaluate the model’s predictive power using metrics such as **Mean Absolute Percentage Error (MAPE)** to assess numerical prediction accuracy, and **Directional Prediction Accuracy (DPA)** to evaluate the model’s ability to make correct predictions in terms of trajectory. Our focus is on understanding how well an HMM trained on real market data can generalize to unseen time periods and whether it captures meaningful temporal dependencies in the stock price behavior.

2 Methodology

This section describes in full detail our modeling choices, the underlying probabilistic graphical model (PGM), the mathematical form of the generative process, how we preprocess and map raw price data into discrete observations, how we parameterize and initialize the HMM, and how we train it via variational inference in Pyro.

2.1 Description of the dataset

The raw data consists on detailed market information for each trading day within the period of 2017 to 2024. Each observation corresponds to one day of trading for a specific stock and includes the following variables:

Variable	Description	Type
Open	Price at market open	Numerical
High	Highest price	Numerical
Low	Lowest price	Numerical
Close	Price at market close	Numerical
Volume	Number of shares traded	Numerical
Price	Adjusted closing price	Numerical

Table 1: Description of stock price variables.

2.2 Data Pre-processing

In this subsection, the raw data is first transformed into scale-invariant fractional features $(\Delta C_t, \Delta H_t, \Delta L_t)$, then each feature is discretized and collapsed into a single integer observation, for the end of the day. Finally, those observations are segmented into overlapping rolling windows to serve as input for the HMM.

2.2.1 Fractional Features

After collection, for each stock and trading day t we compute three scale-invariant returns:

$$\Delta C_t = \frac{C_t - O_t}{O_t}, \quad \Delta H_t = \frac{H_t - O_t}{O_t}, \quad \Delta L_t = \frac{O_t - L_t}{O_t}.$$

Fractional returns are computed by normalizing each price change by the opening price, ensuring scale-invariance across stocks and over time. Expressing movements as percentages mitigates non-stationarity and heteroskedasticity, satisfying the HMM's requirement for time-homogeneous emissions. By including both the day's high and low deviations, we capture intraday volatility patterns that simple close-to-close returns miss, yielding cleaner inputs for regime detection.

2.2.2 Discretization & 3D to 1D Mapping

To convert continuous fractional features into a form suitable for a discrete-output Hidden Markov Model, the following steps are performed:

1. Binning each feature independently:

Discretization is required because the HMM we employ expects integer-valued observations. By binning each feature, we retain the essential variability of intraday movements while reducing the continuous space to a manageable set of symbols. This yields one-based integer indices $x_t \in \{1, \dots, n_x\}$, $y_t \in \{1, \dots, n_y\}$, $z_t \in \{1, \dots, n_z\}$, leading to $N_{\text{bins}} = n_x \times n_y \times n_z$ to be the total number of bins.

2. Mapping to a single integer:

The three indices (x_t, y_t, z_t) are collapsed into a single value n_t :

$$n_t = (z_t - 1) \times (n_x \cdot n_y) + (y_t - 1) \times n_x + (x_t - 1)$$

This produces univariate discrete observations which enables efficient forward-backward inference and is suitable for a standard HMM. Moreover, this step is comparable to [Catello et al. \(2023\)](#)'s method *map3DTo1D* step.

3. Rolling-window segmentation:

The training of the model was based on a fixed number days prior to the prediction, rather than

training on the entire set of previous days sequence¹. As such, we slice it into overlapping latency l windows that shift forward by one day each time. This serves two purposes: it increases the effective number of independent training examples, and it allows the model to learn local regime dynamics over short horizons.

2.3 Parameterisation & Initialization

Prior to training, initial values must be specified for each of the HMM's learnable parameters. These are the start-state distributions, $\pi \in \mathbb{R}^K$, the state-transition matrix, $A \in \mathbb{R}^{K \times K}$, and the state-conditional emission matrix, $B \in \mathbb{R}^{K \times N_{\text{bins}}}$, with K is the number of hidden states, and T is the number of days. Note that the HMM parameters $\pi_{\text{log}}, A_{\text{log}}, B_{\text{log}}$ consist of un-normalized log-probabilities of beginning in each of the four hidden states. These are initialized as follows:

1. Emission logits $B_{\text{log}}^{(0)} \in \mathbb{R}^{K \times N_{\text{bins}}}$.

- (a) The continuous feature matrix $\mathbf{X} \in \mathbb{R}^{T \times 3}$ is clustered via $K\text{Means}(K)$, producing labels $\hat{s}_t \in \{0, \dots, K - 1\}$ and a vector $\text{idxs} \in \{\hat{s}_0, \dots, \hat{s}_{T-1}\}^T$.
- (b) The integer-mapped observations $n = [n_t]$ with $n_t \in \{0, \dots, N_{\text{bins}} - 1\}$ of shape $(T,)$ are counted per cluster:

$$C_{k,n} = \sum_{t=1}^T \mathbf{1}\{\hat{s}_t = k\} \mathbf{1}\{n_t = n\}, \quad C \in \mathbb{N}^{K \times N_{\text{bins}}}$$

- (c) Laplace smoothing ($\alpha = 1$) and row-normalization yield a probability matrix $B^{(0)} \in \mathbb{R}^{K \times N_{\text{bins}}}$:

$$B_{k,n}^{(0)} = \frac{C_{k,n} + 1}{\sum_{m=0}^{N_{\text{bins}}-1} (C_{k,m} + 1)}.$$

- (d) The initial emission logits are then:

$$B_{\text{log}}^{(0)} = \log B^{(0)} \in \mathbb{R}^{K \times n_{\text{bins}}}$$

2. Start-state logits $\pi_{\text{log}}^{(0)} \in \mathbb{R}^K$. These parameters represent the unnormalised log-probabilities of beginning in each of the four hidden states. By drawing

$$\pi_{\text{log},j}^{(0)} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2), \quad j = 1, \dots, K,$$

a nearly uniform prior over initial states is implied once the softmax is applied:

$$\pi_j^{(0)} = \frac{\exp(\pi_{\text{log},j}^{(0)})}{\sum_{i=1}^4 \exp(\pi_{\text{log},i}^{(0)})}$$

This random jitter breaks symmetry and gives the optimizer a direction in which to update π during Baum-Welch.

¹as would also pose the problem of there being a different variables into each prediction

3. Transition logits $A_{\log}^{(0)} \in \mathbb{R}^{K \times K}$. Each row i of this matrix encodes the log-probabilities of transitioning from state i to each of the four states at the next time step. By initializing

$$A_{\log, i, j}^{(0)} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2), \quad i, j = 1, \dots, K,$$

each transition distribution $A_{i, \cdot}^{(0)}$ becomes approximately uniform after softmax. This ensures that no transition is apriori favoured, while still allowing the optimizer to learn meaningful dynamics.

2.4 Training via Baum–Welch (Variational SVI)

Parameter estimation was carried out by maximising the ELBO under a fully-enumerated discrete HMM in Pyro. We optimized the model parameters `pi_logits`, `A_logits` and `B_logits` directly via the ELBO objective, where Pyro computes gradients through the exact marginal likelihood.

The marginal likelihood of a sequence of observations can be computed as the probability of the observed sequence by summing over all possible hidden state sequences:

$$p(\mathbf{x}_{1:T}) = \sum_{\mathbf{z}_{1:T}} p(z_1) \prod_{t=2}^T p(z_t | z_{t-1}) \prod_{t=1}^T p(x_t | z_t)$$

Using the trained parameters, we get:

$$p(\mathbf{x}_{1:T}) = \sum_{\mathbf{z}_{1:T}} \pi_{z_1} \cdot \left(\prod_{t=2}^T A_{z_{t-1}, z_t} \right) \cdot \left(\prod_{t=1}^T B_{z_t}(x_t) \right)$$

During each step of training, we process the data in sliding windows of a fixed length of 10 days. Instead of training on the entire time series as a single sequence, we slice the data into overlapping segments—each of length 10 days, shifting the window forward by one day at a time.

2.5 Prediction

The generative process for the HMM model can be formulated in the following steps, from a stochastic point of view:

1. Sample the first hidden state $z_0 \sim \text{Categorical}(\pi)$.
2. For each time step t in $1, \dots, T$:
 - 2.1. Sample $z_t \sim \text{Categorical}(A_{z_{t-1}})$, the transition from the previous state.
 - 2.2. Sample $x_t \sim \text{Categorical}(B_{z_t})$, to emit the observation from the current hidden state.
3. Repeat until T time steps are generated.

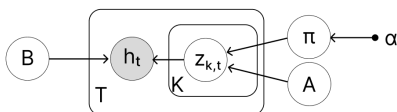


Figure 1: PGM of HMM

This generative process represents a stochastic process, where each state transitions to the next according to a transition probability matrix. At each time step, an observation is stochastically emitted based on the current hidden state, using the associated emission distribution.

For the deterministic approach, we computed the log-likelihood of all possible observations under the emission distribution for the current state and selected the observation with the highest likelihood.

Figure 1 shows a probabilistic graphical model (PGM) for our model. K denotes the number of hidden states and T represents the total number of time steps in each observation sequence.

2.6 Evaluation

To evaluate the accuracy and practical utility of the predictive model, we employ two complementary performance metrics: **Mean Absolute Percentage Error (MAPE)** and **Directional Prediction Accuracy (DPA)**.

We define the following notation²: n_p is the total number of predictions, δ is the identity function, p_i the predicted stock closing price, c_i the actual closing price, and s_i is the stock opening price.

The MAPE is the traditional mean relative error, and as such is calculated between the actual and predicted stock closing prices, accessing numerical prediction accuracy.

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{p_i - c_i}{c_i} \right|$$

The DPA is a metric to evaluate the model’s ability to capture the correct movement direction.³ As such, it is calculated by evaluating the difference in sign between the actual and predicted stock closing prices with respect to the opening price.

$$\text{DPA} = \frac{100\%}{n} \sum_{i=1}^n \delta(\text{sgn}(p_i - s_i), \text{sgn}(c_i - s_i))$$

3 Experiment and Results

3.1 Model’s Parameters

In this project, we utilize financial time series data collected through the **Yahoo Finance API** via the `yfinance` Python package (Yahoo (2025)). The dataset includes **daily historical stock prices** for three major publicly traded companies - **Apple (AAPL)**, **IBM**, and **Dell (DELL)** - covering the period from **January 1st, 2017 to January 1st, 2024**, this leads us to have the number of days $T = 1760$, accounting for the exclusion of weekends and holidays (in New York State of the U.S.A.).

We chose to insert ΔC_t into 50 bins, ΔH_t into 10 bins, ΔL_t into 10 bins. The choice of 50 bins

²in accordance to the Catello’s paper

³Very relevant for investors, as it evaluates the models ability to predict an increase or decrease during the day.

for ΔC_t balances the typically wide range of open-to-close returns against the need to avoid sparsity, whereas 10 bins suffice for the generally smaller high-and-low excursions. As a result, we have that $N_{\text{bins}} = 50 \times 10 \times 10 = 5000$. Note that the edges are chosen dynamically per-stock.

Following Catello's paper, we chose to $K = 4$ to be the number of hidden states, as a result our matrices have the following shapes:

$$\pi_{\log} : (4,), \quad A_{\log} : (4, 4), \quad B_{\log} : (4, 5000).$$

3.2 Model's Performance

3.2.1 Stochastic Prediction

For the stochastic predictions we followed the directional prediction accuracy (DPA) metric in order to verify how reliable our model was in predicting either an increase or a decrease in the stock price at the end of a day. We saw that the DPA for our stochastic predictions was around the 50% mark, which is essentially the equivalent of random guessing the direction of the stock price movement.

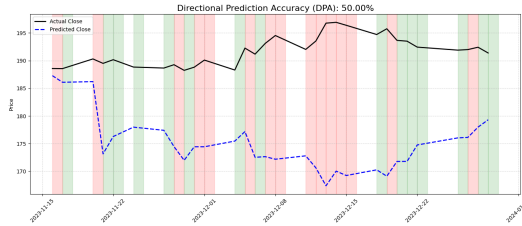


Figure 2: Evaluation of Predictions on APPLE stock

3.2.2 Deterministic Prediction

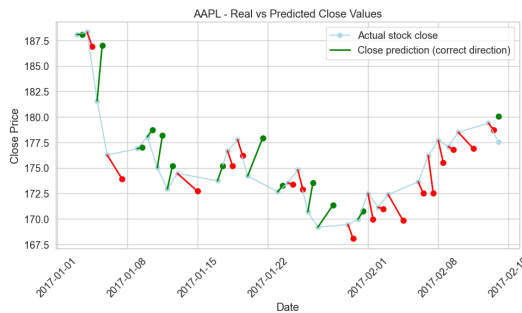


Figure 3: Random Guessing on Stochastic Approach

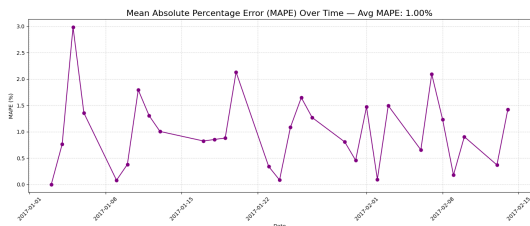


Figure 4: MAPE Evaluation of APPLE stock

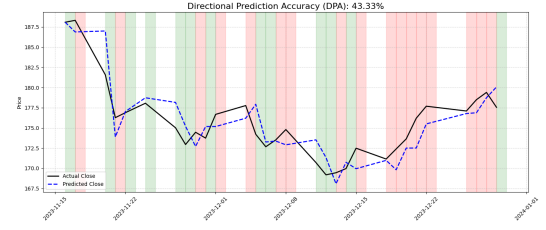
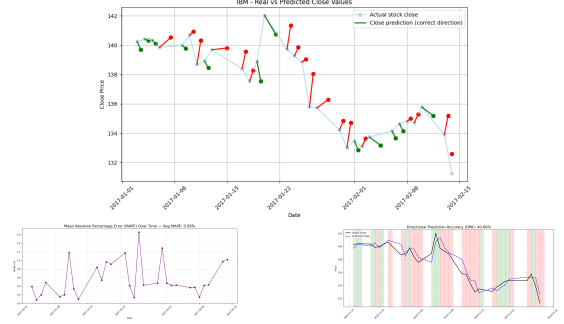


Figure 5: DPA Evaluation of APPLE stock

(a) Predictions vs Observed

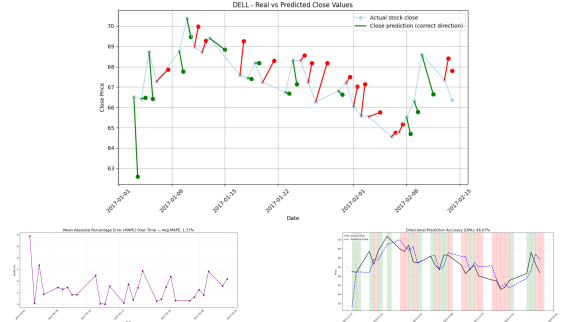


(b) MAPE

(c) DPA

Figure 6: Evaluation of Predictions on IBM stock

(a) Predictions vs Observed



(b) MAPE

(c) DPA

Figure 7: Evaluation of Predictions on DELL stock

Regarding the deterministic predictions, we expanded onto the second metric, the MAPE. We verified from the results in the paper we have followed that for the same data, their model was also getting just above the 40% DPA accuracy and 1% MAPE.

4 Discussion

4.1 Goal Achievement

For Apple, we can see that Figure 3, Figure 4 and Figure 5 show that the final model achieves a Mean Absolute Percentage Error of 1.00% and the Directional Prediction Accuracy is only 43.33%. For IBM, Figure 6 shows the MAPE of 0.56% and the DPA is

only 40.0%. And for Dell, **Figure 7** shows the MAPE of 1.37% and the DPA is only 46.67%.

This low MAPE reflects strong performance in approximating the true numerical values; indicating that, on average, its predicted values are very close to the actual stock prices in magnitude. However, the DPA is below the 50% threshold expected from random guessing.

This means that the model adequately the value of the stock, it fails to consistently predict whether the stock will go up or down. In practical terms, this limits the model’s usefulness in decision-making contexts—such as trading, especially over longer time periods.

The results collected underline the possible existence of external factors that are not captured by the model’s posterior distribution in order to sample more accurate observations. An extension of the model could be beneficial in capturing extra context for a better posterior distribution.

4.2 Future work

Predicting the close price of a stock is not a trivial task. Despite the good results obtained in this short project, further work could be conducted in order to extend and possibly explore the viability of more complex models.

One of the main drivers of stock prices in the open market are publicly available news, which have historically shown to impact substantially the movements of stock prices over time.

This model could be extended to include a topic model by incorporating latent or observed semantic information derived from financial news. For each time step t , we assume access to a document (e.g., news articles from the previous 24 hours) from which we extract a topic distribution ϕ_t , either via a pre-trained topic model (such as LDA) or jointly during training. The emission distribution is then conditioned not only on the hidden state z_t , but also on ϕ_t , allowing the model to capture contextual information from external

text sources. Consequently, the emission becomes:

$$x_t \sim p(x_t | z_t, \phi_t)$$

This modification of the model introduces an additional dependency in the probabilistic graphical model (PGM), enriching the observation model with external semantic features. This implementation potentially improving predictive performance in highly volatile or news-driven environments.

Contributions

Since the early stages of conceptualization and development of this project, our group has taken a balanced workload approach where each team member would provide their insights and contribute to the discussed matter equally.

Nevertheless, as the project advanced, each member naturally taken different responsibilities and focusing more in some tasks whilst still providing aid and their thoughts to other work being done. This way, while Rita and Lydia focused more in tasks as the PGM, data processing and parameterisation and initialization, Afonso and Patrick focused more in the model creation, training, prediction and results.

Nonetheless, as mentioned above, this was a combined effort, where all member were aiding and inputting into each task until the final conclusion was achieved.

References

- L. Catello, L. Ruggiero, L. Schiavone, and M. Valentino. Hidden markov models for stock market prediction. *arXiv preprint arXiv:2310.03775*, 2023. URL <https://arxiv.org/abs/2310.03775>.
- Yahoo. yfinance: Download market data from yahoo! finance’s api. <https://pypi.org/project/yfinance/>, 2025. Accessed: 2025-05-26.